

Day 4 — Recursion

Faculty: Vipul Tembulwar

Base condition

Direct & indirect recursion

Call stack & memory

Pros / cons

Complexity of recursive functions

Recursion is a method solving a problem by calling itself on a smaller version of the same problem. Trees, sorting (merge sort, quick sort), graphs (depth-first search) and algorithm design (divide and conquer, backtracking, dynamic programming) all depend on it. Today we learn to write recursion, to see it on the call stack, and to calculate its cost.

1. What is recursion?

DEFINITION Recursion

A technique in which a method solves a problem by calling **itself** on one or more **smaller** instances of the same problem, until it reaches an instance small enough to solve directly.

Every correct recursive method has exactly two parts:

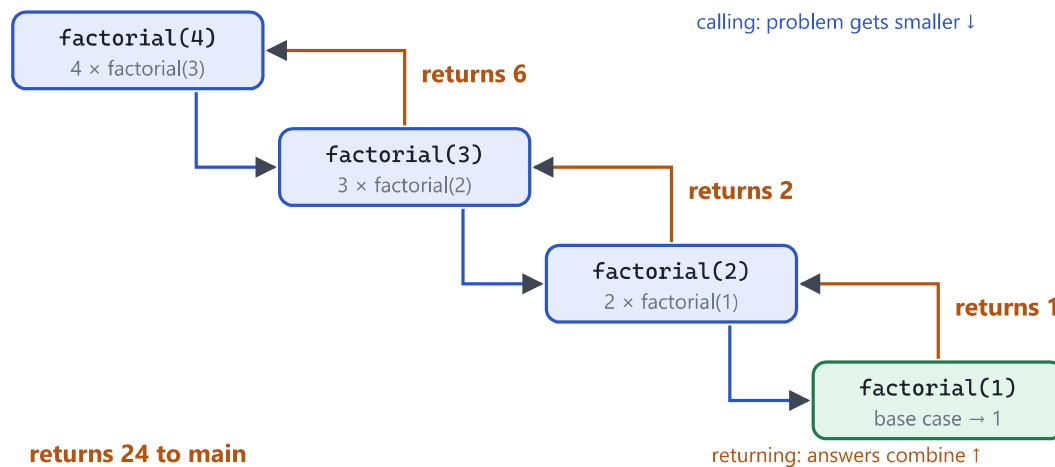
1. **Base case** — the smallest input, answered directly **without** recursion. It stops the process.
2. **Recursive case** — breaks the problem into a smaller one, calls itself, and uses the result.

JAVA RecursionBasics.java

```
static long factorial(int n) {  
    if (n ≤ 1) return 1;           // 1. base case  
    return n * factorial(n - 1);   // 2. recursive case: a smaller problem  
}
```

Many definitions in mathematics are already recursive, so the code reads like the definition:

Problem	Recursive definition	Base case
$n!$	$n \times (n-1)!$	$0! = 1$
sum of <code>a[0..n-1]</code>	sum of <code>a[0..n-2]</code> + <code>a[n-1]</code>	empty array $\rightarrow 0$
x^n	$x \times x^{n-1}$	$x^0 = 1$
sum of digits of 4721	1 + digits of 472	a single digit
gcd(a, b)	gcd(b, a mod b)	gcd(a, 0) = a
a linked list	a node followed by a (smaller) list	<code>null</code>



Winding (blue): each call waits for a smaller one. Unwinding (orange): each call finishes its multiplication.

TIP The "leap of faith"

When writing `factorial(n)`, **don't trace every level in your head**. Assume `factorial(n-1)` already returns the right answer, and ask only: "How do I use it to get `factorial(n)`?" Then check the base case. If both are right, the whole method is right. This is proof by **mathematical induction**.

How to write any recursive method — 3 questions

1. **What is the smallest version of this problem, and what is its answer?** That is your base case.

2. **How can I make the problem smaller?** For example, `n-1` , `n/2` , the rest of the string, or the next node.
3. **If the smaller problem is solved, how do I build my answer from it?**

Try it on "reverse a string":

1. A string of length 0 or 1 is its own reverse.
2. Remove the first character.
3. `reverse(rest) + firstChar` .

JAVA

```
static String reverse(String s) {
    if (s.length() ≤ 1) return s;
    return reverse(s.substring(1)) + s.charAt(0);    // "RECURSION" → "NOISRUCER"
}
```

More examples from `RecursionBasics.java`:

JAVA

```
static boolean isPalindrome(String s, int lo, int hi) {
    if (lo ≥ hi) return true;                // 0 or 1 chars left
    if (s.charAt(lo) ≠ s.charAt(hi)) return false;    // mismatch: stop early
    return isPalindrome(s, lo + 1, hi - 1);    // check the middle
}

static int gcd(int a, int b) {                // Euclid, 300 BC
    if (b == 0) return a;
    return gcd(b, a % b);                    // gcd(252, 105) = 21
}

static int binarySearch(int[] a, int key, int lo, int hi) {
    if (lo > hi) return -1;                    // empty range: not found
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == key) return mid;            // found
    return key < a[mid] ? binarySearch(a, key, lo, mid - 1)
                        : binarySearch(a, key, mid + 1, hi);
}
```

```
        : binarySearch(a, key, mid + 1, hi);  
    }
```

**QUICK
CHECK**

Write `sumDigits(4721)` recursively. What are the base case and the recursive case?

[Show answer](#)

2. The base condition

The base case is what makes recursion **finite**. Two things must be true:

1. The base case exists and returns **without** recursing.
2. Every recursive call moves the input **closer** to the base case.

If either fails, the calls never stop, the stack fills up, and Java throws `StackOverflowError`.
Output of `StackDepth.java` on one machine:

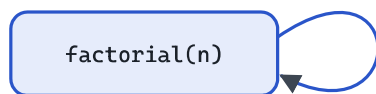
Program	What went wrong	Calls before the crash
<code>noBaseCase()</code>	no base case at all	23,306
<code>bigFrame(...)</code> (six <code>long</code> parameters and six locals)	same, but each frame is bigger	4,539
<code>wrongBase(-1)</code>	the base case is <code>n == 0</code> , but <code>n = -1</code> moves away from it	9,639

WATCH OUT Base-case bugs to watch for

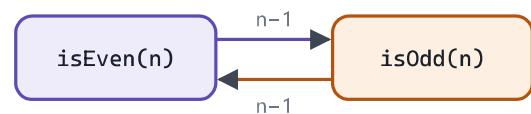
- `if (n == 0)` when `n` can be negative → use `if (n ≤ 0)` or validate the input.
- Making the input smaller in the wrong place: `f(n)` calls `f(n)` instead of `f(n - 1)`.
- Two base cases needed but only one written: Fibonacci needs **both** `fib(0)` and `fib(1)`.

- With objects: a cycle, such as `a.manager = b`; `b.manager = a`; plus a recursive `toString()`, never reaches `null`.

3. Direct and indirect recursion



Direct: calls itself



Indirect: $A \rightarrow B \rightarrow A$

Indirect (mutual) recursion is still recursion: the cycle must also reach a base case

Kind	Meaning	Example
Direct	a method calls itself	<code>factorial</code> , <code>binarySearch</code>
Indirect (mutual)	A calls B, B calls A	<code>isEven</code> ↔ <code>isOdd</code> , and parsers (expression → term → factor → expression)
Linear	at most one recursive call per invocation	<code>factorial</code> , <code>sum</code> , <code>reverse</code>
Tree (multiple)	two or more recursive calls per invocation	<code>fib</code> , Tower of Hanoi, merge sort
Head	the recursive call comes first, and the work happens while returning	print a list backwards
Tail	the recursive call is the last action	<code>gcd</code> , <code>binarySearch</code>

JAVA `DirectIndirect.java`

```

static boolean isEven(int n) { return n == 0 ? true : isOdd(n - 1); }
static boolean isOdd(int n) { return n == 0 ? false : isEven(n - 1); }
  
```

```

static void head(int n) {                                // prints 1 2 3 4 5
    if (n == 0) return;
    head(n - 1);                                         // call first ...
    System.out.print(n + " ");                           // ... work while returning
}

static void tail(int n) {                                // prints 5 4 3 2 1
    if (n == 0) return;
    System.out.print(n + " ");                           // work first ...
    tail(n - 1);                                         // ... call last
}

```

INSIDE JAVA.UTIL Java does not optimise tail recursion

Some languages (Scheme, Scala with `@tailrec`, Kotlin with `tailrec`) turn a tail call into a jump, so tail recursion uses no extra stack. **The JVM does not**, so a tail-recursive Java method still uses one frame per call and can still overflow. Luckily, any tail recursion converts mechanically into a loop:

JAVA

```

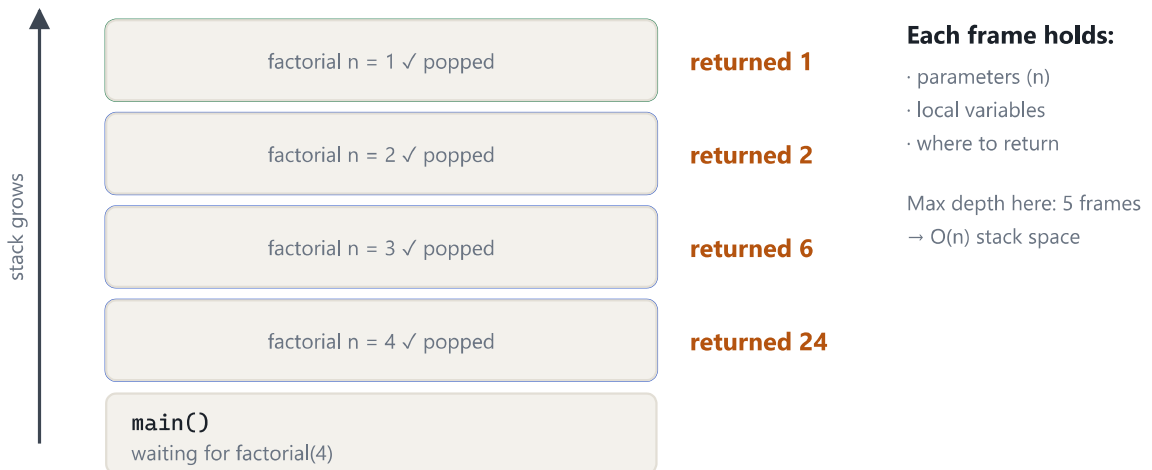
static long factTail(int n, long acc) {                 // tail recursive
    if (n ≤ 1) return acc;
    return factTail(n - 1, acc * n);
}

static long factLoop(int n) {                           // the same thing as a loop
    long acc = 1;
    while (n > 1) { acc *= n; n--; }
    return acc;
}

```

4. Memory: what happens on the call stack

Every time a method is called, the JVM pushes a **stack frame** onto the current thread's **call stack**. The frame holds the method's parameters, its local variables and the place to return to. When the method returns, its frame is popped. This is the **stack** data structure from Day 2, working inside the JVM.



Frames are pushed on every call and popped on every return. The frame on top is the one running.

Output of `CallStackTrace.java` :

TEXT

```
→ factorial(4)   frame pushed, stack depth = 1
  → factorial(3)   frame pushed, stack depth = 2
    → factorial(2)   frame pushed, stack depth = 3
      → factorial(1)   frame pushed, stack depth = 4
        ← factorial(1) returns 1   frame popped
      ← factorial(2) returns 2   frame popped
    ← factorial(3) returns 6   frame popped
  ← factorial(4) returns 24   frame popped
```

Key points

- Each call has its **own copy** of `n`. The four `n` variables live in four different frames and never interfere.
- A frame stays on the stack while it **waits** for the call it made. That is why `factorial(n)` needs **O(n) stack space** even though it creates no arrays.

- The stack is **small**: by default roughly 512 KB to 1 MB per thread, tunable with `java -Xss4m`. Objects created with `new` live on the **heap**, which is much larger. Only the references to them sit in the frame.
- An exception's **stack trace** is a printout of the call stack at the moment it was thrown.

TRY IT See it in your IDE

Put a breakpoint on `return 1;` inside `factorial` and run in debug mode. The **Frames** (or **Call Stack**) panel shows `factorial` four times, each with its own `n`. Step out one level at a time and watch the frames disappear.

5. Pros and cons of recursion

✓ Advantages	✗ Disadvantages
Short, readable code that matches the problem's definition	Every call costs time (pushing a frame, the call, the return) and stack memory
Natural for recursive structures : trees, folders, nested JSON or XML, grammars	Deep recursion → <code>StackOverflowError</code> (Java has no tail-call optimisation)
Natural for divide and conquer : merge sort, quick sort, binary search	Easy to write exponential code by accident, such as naive <code>fib</code>
Easy to explore choices (backtracking: subsets, permutations, mazes)	Harder to debug and trace for beginners
Correctness can be proved by induction	Usually slower than an equivalent loop in Java

Recursion vs iteration — the same function, three ways

Output of `FibonacciCompare.java`:

n	fib(n)	naive recursive calls	naive time	memoised calls	loop
20	6,765	21,891	1 ms	39	instant
30	8,32,040	26,92,537	4 ms	59	instant

n	fib(n)	naive recursive calls	naive time	memoised calls	loop
40	10,23,34,155	33,11,60,281	519 ms	79	instant

JAVA

```

static long naive(int n) {                                // O(2^n) time, O(n) stack
    if (n < 2) return n;
    return naive(n - 1) + naive(n - 2);
}

static long memo(int n, long[] cache) {                  // O(n) time, O(n) space
    if (n < 2) return n;
    if (cache[n] ≠ 0) return cache[n];                  // already solved: reuse it
    return cache[n] = memo(n - 1, cache) + memo(n - 2, cache);
}

static long iterative(int n) {                           // O(n) time, O(1) space
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}

```

The naive version is slow because of recursion that **repeats the same work** (Day 1's fib tree), not because of recursion itself. Remembering answers (**memoisation**) keeps the recursive shape but removes the repeats. We do this with a hash table on Day 8 and generalise it as **dynamic programming** on Day 11.

TIP When to choose recursion

Choose recursion when the **data or the problem is naturally recursive** (trees, graphs, divide and conquer, backtracking) and the depth stays small, around $\log n$ or a few thousand levels at most. Choose a loop for simple linear repetition over large n , such as walking a 10-lakh-node list. `RecursiveLinkedList.java` shows `length()` of a 10,00,000-node list overflowing the stack.

6. Complexity of recursive functions

Time = (number of calls) × (work done in each call, excluding its recursive calls).

Space = maximum recursion depth × space per frame (plus any data the calls create).

To count calls, we write a **recurrence relation**, $T(n)$ defined in terms of T of smaller inputs, and solve it.

Method 1 — unrolling (substitution)

Factorial: $T(n) = T(n-1) + c$

TEXT

$$\begin{aligned} T(n) &= T(n-1) + c \\ &= T(n-2) + 2c \\ &= T(n-3) + 3c \\ &\dots \\ &= T(1) + (n-1)c \quad \rightarrow \quad O(n) \end{aligned}$$

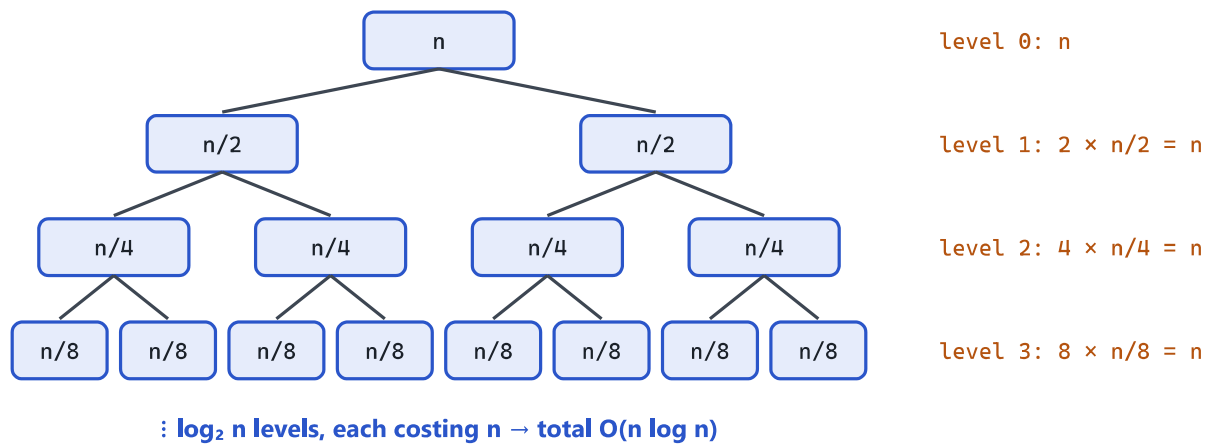
Binary search: $T(n) = T(n/2) + c$

TEXT

$$T(n) = T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = \dots = T(1) + (\log_2 n) \cdot c \quad \rightarrow \quad O(\log n)$$

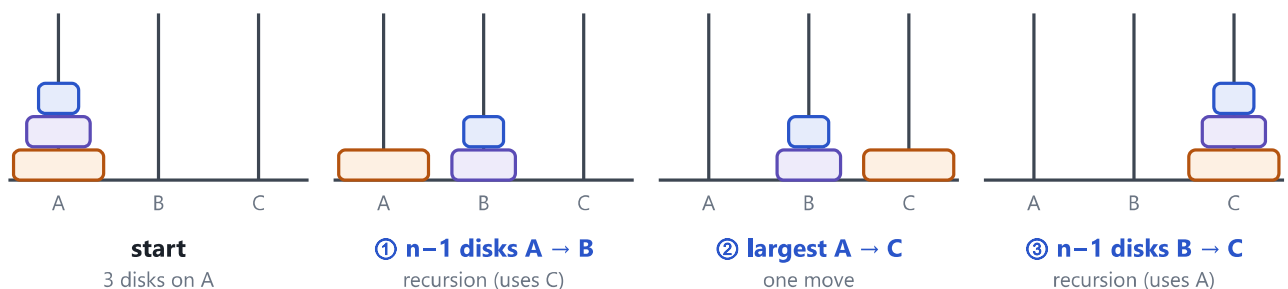
Method 2 — the recursion tree

Draw the calls as a tree, write the work done at each node, and add up the work **level by level**.



Add the work level by level. This is exactly merge sort (Day 7).

Tower of Hanoi — an exponential recurrence



$\text{hanoi}(n) = \text{hanoi}(n-1) + 1 \text{ move} + \text{hanoi}(n-1)$, so $T(n) = 2T(n-1) + 1 = 2^n - 1$ moves

JAVA TowerOfHanoi.java

```
static void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);    // ① move n-1 disks out of the way
    System.out.println("move disk " + n + " : " + from + " → " + to);    // ②
    hanoi(n - 1, via, to, from);    // ③ put them back on top
}
```

$T(n) = 2T(n-1) + 1$ unrolls to $2^n - 1$. For $n = 64$ disks that is 18,446,744,073,709,551,615 moves. At one move per second, that would take about **585 billion years**.

Common recurrences — memorise this table

Recurrence	Example	Time	Stack depth
$T(n) = T(n-1) + O(1)$	factorial, sum, list length	$O(n)$	$O(n)$
$T(n) = T(n/2) + O(1)$	binary search, fast power	$O(\log n)$	$O(\log n)$
$T(n) = T(n-1) + O(n)$	selection sort written recursively	$O(n^2)$	$O(n)$
$T(n) = 2T(n/2) + O(n)$	merge sort	$O(n \log n)$	$O(\log n)$
$T(n) = 2T(n/2) + O(1)$	tree traversal, max of an array by halves	$O(n)$	$O(\log n)$ for a balanced tree
$T(n) = 2T(n-1) + O(1)$	Tower of Hanoi, naive subsets	$O(2^n)$	$O(n)$
$T(n) = T(n-1) + T(n-2) + O(1)$	naive Fibonacci	$O(\varphi^n) \approx O(1.618^n)$, written $O(2^n)$	$O(n)$

Output of `RecurrenceCounter.java` confirms the formulas:

n	$T(n-1)+1$	$T(n/2)+1$	$2T(n/2)+n$	$n \log_2 n$
1,024	1,024	10	10,240	10,240
4,096	4,096	12	49,152	49,152
8,192	8,192	13	1,06,496	1,06,496

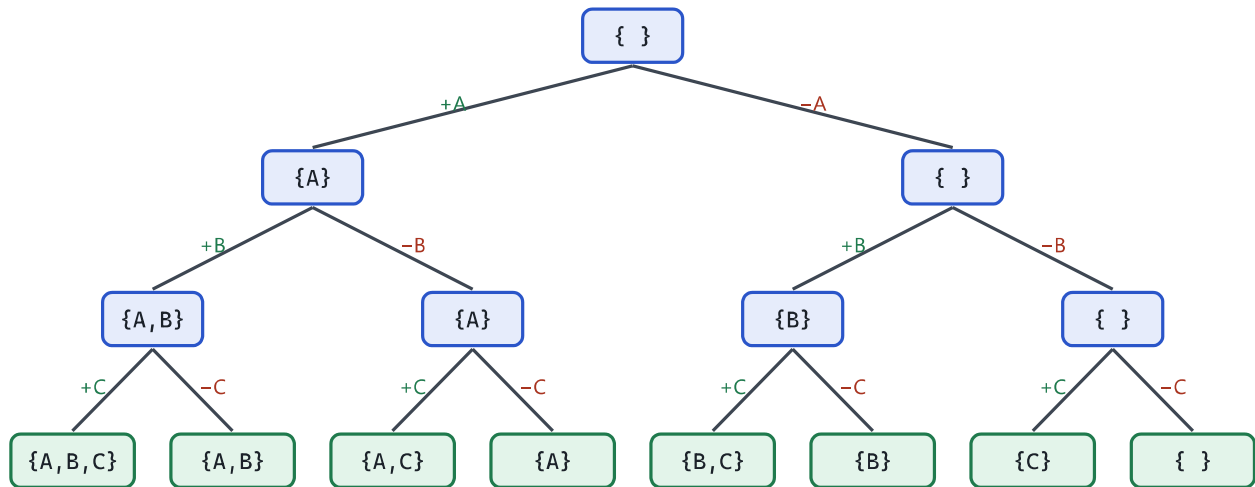
QUICK CHECK

Fast power computes `half = power(x, n/2)` once and returns `half * half`. What if we wrote `power(x, n/2) * power(x, n/2)` instead?

Show answer

7. Recursion that explores choices

Some problems ask for **all** possibilities. Recursion handles them naturally: make one choice, recurse on the rest, then try the next choice.



At each level we decide one item: take it (+) or leave it (-). 3 decisions $\rightarrow 2^3 = 8$ leaves.

JAVA SubsetsPermutations.java

```

static void subsets(String items, int i, String chosen, List<String> out) {
    if (i == items.length()) { out.add("'" + chosen + "'"); return; } // all done
    subsets(items, i + 1, chosen + items.charAt(i), out); // take
    subsets(items, i + 1, chosen, out); // leave
}

static void permutations(String prefix, String remaining, List<String> out) {
    if (remaining.isEmpty()) { out.add(prefix); return; }
    for (int i = 0; i < remaining.length(); i++) // choose
        permutations(prefix + remaining.charAt(i),
            remaining.substring(0, i) + remaining.substring(i + 1), out)
}

```

- Subsets of n items: 2^n . Permutations: $n!$. Both explode quickly: 15 letters have 1.3 trillion permutations.
- On Day 11, **backtracking** adds one idea: stop exploring a branch as soon as it cannot lead to a valid answer (for example, N-Queens or Sudoku).

8. Recursion in everyday Java

INSIDE JAVA.UTIL You already depend on recursion

- **File systems** are recursive: a folder contains files and folders. `countFiles(dir)` calls itself on each sub-folder. `Files.walk` does the same job for you.
- **Linked structures**: `TreeMap` walks its red-black tree (Day 6), and JSON and XML libraries parse nested objects recursively.
- **Sorting**: `Arrays.sort` for primitives is a recursive **quicksort**, and objects use **TimSort**, a merge-sort variant (Day 7).
- `toString()`, `equals()` and `hashCode()` on nested collections call themselves on every element. The JDK guards `toString()` against a list that contains itself (it prints `(this Collection)`), but `hashCode()` on the same list overflows the stack. Output of `JavaRecursionGotchas.java`:

TEXT

```
list containing itself, toString(): [x, (this Collection)]
list containing itself, hashCode(): StackOverflowError
Employee cycle in toString(): StackOverflowError
```

Recursion on a linked list

A linked list is itself a recursive definition: *a list is either empty (`null`) or a node followed by a list*. So list algorithms become two-line recursive methods. This also answers the **Day 2 home challenge**:

JAVA `RecursiveLinkedList.java`

```
static int length(Node n) {
    if (n == null) return 0;
    return 1 + length(n.next);
}

static void printBackward(Node n) {           // head recursion
    if (n == null) return;
```

```

    printBackward(n.next);
    System.out.print(n.data + " ");           // 40 30 20 10
}

static Node reverse(Node head) {
    if (head == null || head.next == null) return head; // 0 or 1 node
    Node newHead = reverse(head.next);               // trust: the rest is
    head.next.next = head;                             // my old next now poi
    head.next = null;                                   // I am the new tail
    return newHead;
}

```

9. Summary

- **Recursion** = a base case + a recursive case on a **smaller** input. Trust the recursion, and check the base case.
- A missing base case, or input that does not shrink towards it, gives **StackOverflowError**.
- **Direct** (calls itself) vs **indirect** ($A \rightarrow B \rightarrow A$). **Head** vs **tail**. **Linear** vs **tree** recursion.
- Each call gets its own **stack frame** (parameters, locals, return point). Depth $\times$ frame size = stack space. The Java stack is small, and there is **no tail-call optimisation**.
- **Pros**: clarity for recursive data and divide and conquer. **Cons**: call overhead, stack limits, and accidental exponential work.
- **Complexity**: write the recurrence, then solve it by unrolling or with a recursion tree. Memorise the common recurrences table.
- Recursion that repeats sub-problems $\rightarrow$ **memoise** (Day 8, Day 11).

10. Lab sheet — Day 4

The code files for these tasks will be shared by your faculty. Run a file directly with `java FileName.java` (Java 11+), or open it in your IDE.

LAB Task 1 — Write these recursively

For each one, write the base case and the recursive case as comments **before** coding:

1. `countOccurrences(int[] a, int n, int x)`
2. `isSorted(int[] a, int n)`
3. `maxOf(int[] a, int lo, int hi)` by splitting the array into two halves
4. `printTriangle(n)`, which prints n stars, then $n-1$, ..., then 1
5. `toBinary(int n)`, which returns the binary string of n

LAB Task 2 — Trace the call stack

Add the indentation trace from `CallStackTrace.java` to your `sumDigits` and `binarySearch`. Run them on `sumDigits(98765)` and on a search for a missing key in a 16-element array. How deep does each stack get? Compare with n and $\log_2 n$.

LAB Task 3 — Direct and indirect recursion

1. Write mutually recursive `isEven / isOdd` and find the smallest n that causes `StackOverflowError` on your machine.
2. Run `StackDepth.java` normally and with `java -Xss8m StackDepth.java`. Record both results.

LAB Task 4 — Complexity of recursive functions (syllabus: "function complexity during recursion")

For each method, write the recurrence, solve it, and then **verify it with a call counter** as in `RecurrenceCounter.java`: `powerSlow`, `powerFast`, `binarySearch`, `hanoi`, `fib` (naive), and `maxOf` from Task 1.

LAB Task 5 — Recursion vs iteration

Rewrite `factorial`, `gcd`, `binarySearch` and `reverse` (string) as loops. Then write a recursive `length()` for a linked list and find the list size at which it overflows. Explain in two lines which version you would use in production, and why.

TRY IT Home challenge

1. Print all **binary strings of length n** that have no two consecutive 1s.
2. **Flood fill**: given a 2-D `char` grid, recursively recolour every cell connected to (r, c). This is exactly depth-first search (Day 9).
3. Write `permutations` for a string with **repeated** letters (like `"AAB"`) that prints each distinct arrangement once.

11. Quick quiz

QUICK CHECK 1. What are the two essential parts of every recursive method?[Show answer](#)**QUICK CHECK** 2. What does `mystery(5)` print?

JAVA

```
static void mystery(int n) {  
    if (n ≤ 0) return;  
    mystery(n - 2);  
    System.out.print(n + " ");  
}
```

[Show answer](#)**QUICK CHECK** 3. What are the time complexity and the stack space of recursive binary search on n elements?[Show answer](#)**QUICK CHECK** 4. Solve $T(n) = 2T(n/2) + n$.[Show answer](#)

**QUICK
CHECK**

5. Your recursive method works for $n = 1,000$ but crashes with `StackOverflowError` for $n = 1,00,000$. The logic is correct. What are your options?

[Show answer](#)

Next class — Day 5: Trees & Binary Search Trees. Every tree algorithm tomorrow is recursive, so today's skills will be used immediately.